*Capable People, Capable Communities*

## Micro-Credentials in Software Development

### Certificate in Software Development

### MCSD51

# Project Proposal

# “FamilyJoy”

**DongHan WU**

**November 2025**

# Contents


1. INTRODUCTION ..... 3
2. SCOPE ..... 3
3. USER STORIES AND USER CASES ..... 4
4. ETHICAL AND CULTURAL IMPACT ..... 8
5. SDLC ..... 9
6. PROJECT TIMELINE ..... 10
7. DIAGRAMS ..... 12
8. REFERENCES ..... 13

# 1. Introduction

Modern families often struggle to balance work demands with effective child guidance. Traditional methods like verbal reminders frequently lead to conflict and fail to consistently develop children's responsibility.

FamilyJoy addresses this through a gamified web application that transforms family management. Parents create customized tasks while children earn points for completed activities, redeemable for rewards. This structured approach promotes positive communication and responsibility.

**Key Stakeholders:**

- Parents: Create and manage family tasks and rewards
- Children (6-14): Complete tasks and redeem earned points
- System Administrator: Maintains application security and performance

The project aims to deliver a secure platform that reduces family conflict through structured task management and positive reinforcement, while demonstrating robust development practices.

# 2. Scope

FamilyJoy will be developed as a Node.js/Express web application with EJS templating and MySQL database. The project scope follows MoSCoW prioritization to ensure delivery of core functionality.

**Must Have**

- User authentication and family management (registration, family creation, secure linking)
- Task management system (create/edit/assign tasks with point values)
- Task approval workflow (completion marking, parental approval, point allocation)
- Points and reward system (balance tracking, reward catalogue, redemption)
- Administrator functions (user management, password reset)

**Should Have**

- Mobile-responsive design
- Task categorization and filtering
- Visual progress indicators

**Could Have**

- Task scheduling

- Achievement badges
- Family messaging

**Won't Have**

- Virtual environments or social features
- Email notifications or mobile apps
- Advanced analytics or third-party integrations

The application will deliver six core pages: Login, Registration, Parent Dashboard, Child Dashboard, Reward Store, and Administrator Dashboard, ensuring focused development of essential features within project constraints.

## 3. User stories and user cases

**Parent Stories**

- Create family group with invitation code to link children's accounts
- Create tasks with descriptions and point values for clear expectations
- Assign tasks to specific children for appropriate responsibilities
- Review and approve completed tasks to ensure quality before awarding points
- Create reward catalogue with point costs to provide clear goals

**Child Stories**

- Join family using provided code to participate in reward system
- View all assigned tasks in one place to track responsibilities
- Mark tasks complete to earn points for finished work
- Browse and redeem rewards using points to enjoy achievements
- View point balance and history to track progress over time

**Administrator Stories**

- Manage user accounts to maintain system security
- Reset user passwords to assist with credential recovery
- Monitor system activity to ensure optimal performance

**Use Cases:**

1. Authentication Module
   - Log In

     Actor: Parent, Child, Administrator

     Goal: Access system using credentials.

Description: User enters username/password. System validates and redirects to appropriate dashboard.

- Register Parent Account

  Actor: Parent

  Goal: Create new parent account.

  Description: New user provides basic info to register as parent. System creates account and initiates family setup.

- Register Child Account

  Actor: Child

  Goal: Create child account linked to family.

  Description: New user provides info and valid Family Code. System creates account linked to specified family.

2. Family Management Module
   - Create Family Group

     Actor: Parent

     Goal: Establish new family unit.

     Description: Parent initiates family creation. System creates family group, makes parent admin, and generates Family Code.

   - Generate Family Code

     Actor: Parent

     Goal: Obtain invitation code for children.

     Description: Parent requests family code. System displays/regenerates unique code for sharing.

   - Join Family

     Actor: Child

     Goal: Link account to existing family.

     Description: Child enters valid Family Code. System validates and adds child to family group.

3. Task Management Module
   - Create Task

     Actor: Parent

     Goal: Define new rewardable task.

Description: Parent defines task title, description, and point value. System saves task definition.

- Assign Task

  Actor: Parent

  Goal: Assign task to children.

  Description: Parent selects children for task assignment. System links task to children and adds to their lists.

- Edit Task

  Actor: Parent

  Goal: Modify existing task details.

  Description: Parent updates task information. System saves changes to task definition.

- View Task Dashboard

  Actor: Child

  Goal: Review assigned tasks and status.

  Description: System displays child's task list with current status on dashboard.

- Submit Task for Approval

  Actor: Child

  Goal: Request points for completed task.

  Description: Child marks task complete. System updates status to "Pending Approval" and notifies parent.

4. Approval Module
   - Review Approval Queue

     Actor: Parent

     Goal: Monitor tasks awaiting approval.

     Description: System shows parent list of tasks submitted by children for review.

   - Approve Task

     Actor: Parent

     Goal: Confirm completion and award points.

     Description: Parent approves completed task. System marks task approved and awards points to child.

   - Reject Task

Actor: Parent

Goal: Return task for rework.

Description: Parent rejects task completion. System resets task status and notifies child.

- Award Points

  Actor: System

  Goal: Update child's point balance.

  Description: System automatically adds points to child's balance when task is approved.

5. Rewards Module
   - Create Reward

     Actor: Parent

     Goal: Add new reward to catalogue.

     Description: Parent defines reward details and point cost. System adds reward to family catalogue.

   - Browse Reward Store

     Actor: Child

     Goal: View available rewards.

     Description: System displays available rewards with point costs for child to browse.

   - Redeem Reward

     Actor: Child

     Goal: Exchange points for reward.

     Description: Child selects reward to redeem. System checks points, deducts cost, and notifies parent.

   - Fulfil Reward

     Actor: Parent

     Goal: Complete reward process.

     Description: Parent marks redeemed reward as fulfilled after providing it. System records completion.

6. Administrator Module
   - Manage User Accounts

     Actor: Administrator

Goal: Oversee and manage all system users

Description: Administrator views user list, can enable/disable accounts, and update user roles.

- Reset User Password

  Actor: Administrator

  Goal: Assist users who forgot credentials

  Description: administrator selects user and triggers password reset. System generates temporary password.

- View System Dashboard

  Actor: Administrator

  Goal: Monitor system health and usage

  Description: System displays metrics on user activity, family counts, and task completion rates.

## 4. Ethical and Cultural Impact

**Ethical Implications**

FamilyJoy complies with NZ's Privacy Act 2020, protecting family data through encryption and access controls. Parents control data usage through clear consent mechanisms. The design promotes positive parent-child communication through task-reward interactions while respecting children's autonomy and avoiding surveillance.

**Cultural Considerations**

The application supports NZ's diverse families, including Māori whānau and Pacific Islander families, through customizable tasks and rewards that respect different cultural values. The interface emphasizes inclusivity and promotes non-material rewards, aligning with NZ's focus on family wellbeing across cultures.

# 5. SDLC

**Selected Methodology: Agile - Scrum Framework**

We have chosen the Scrum framework to develop the FamilyJoy project. This lightweight, iterative agile methodology is particularly suitable for projects like ours with clear requirements but needing flexibility during implementation. It enables us to maintain functionality quality while remaining responsive to feedback.

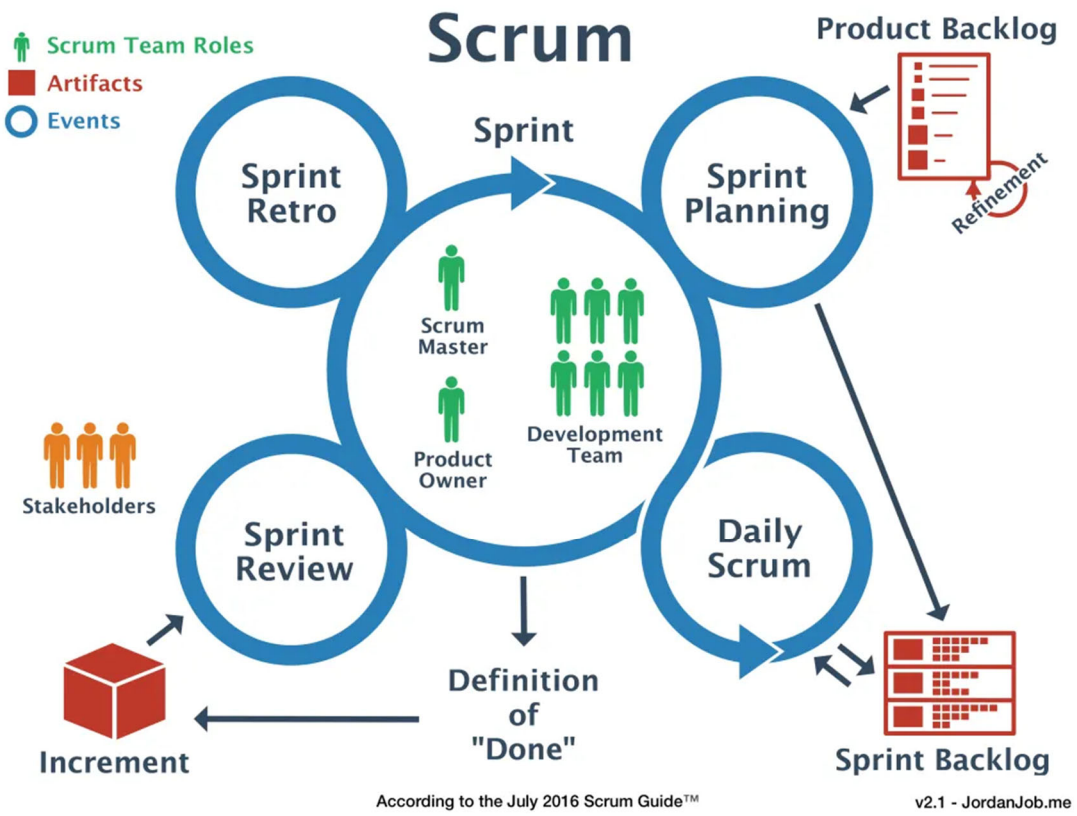



*Figure 1*

*Illustration from How Scrum helped turn around our development process by J. Wu (2019), Medium (https://medium.com/@jw207427/how-scrum-help-turn-around-our-development-process-dac6ff7c700). Copyright 2019 by the author.*

**Implementation Plan:**

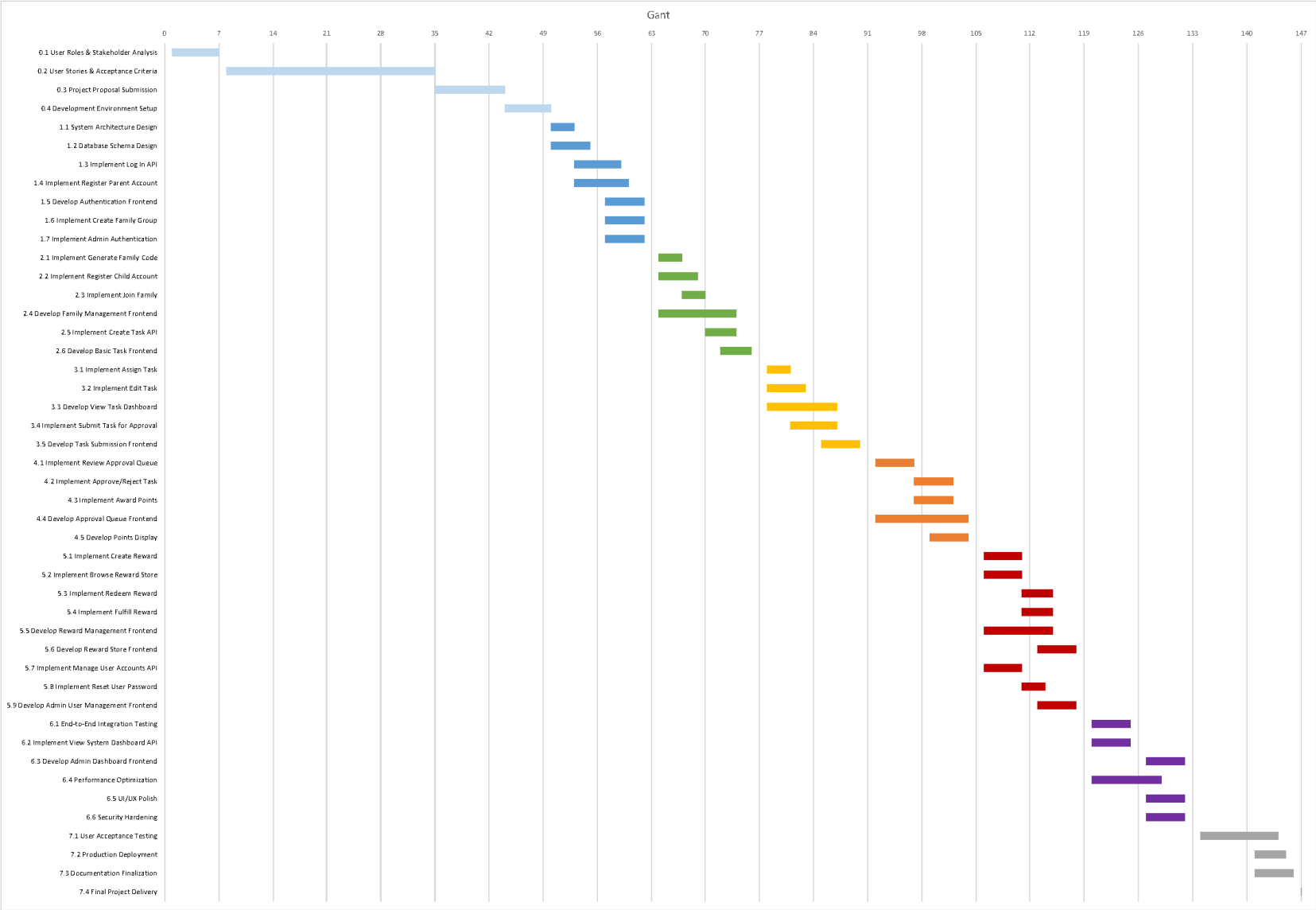
- **Roles**: The team combines Product Owner and Development roles for backlog management and implementation
- **Artifacts**: MoSCoW-prioritized user stories form the Product Backlog
- **Cycles**: Two-week Sprints with planning, daily check-ins, reviews, and retrospectives

**Reason for Choice:**

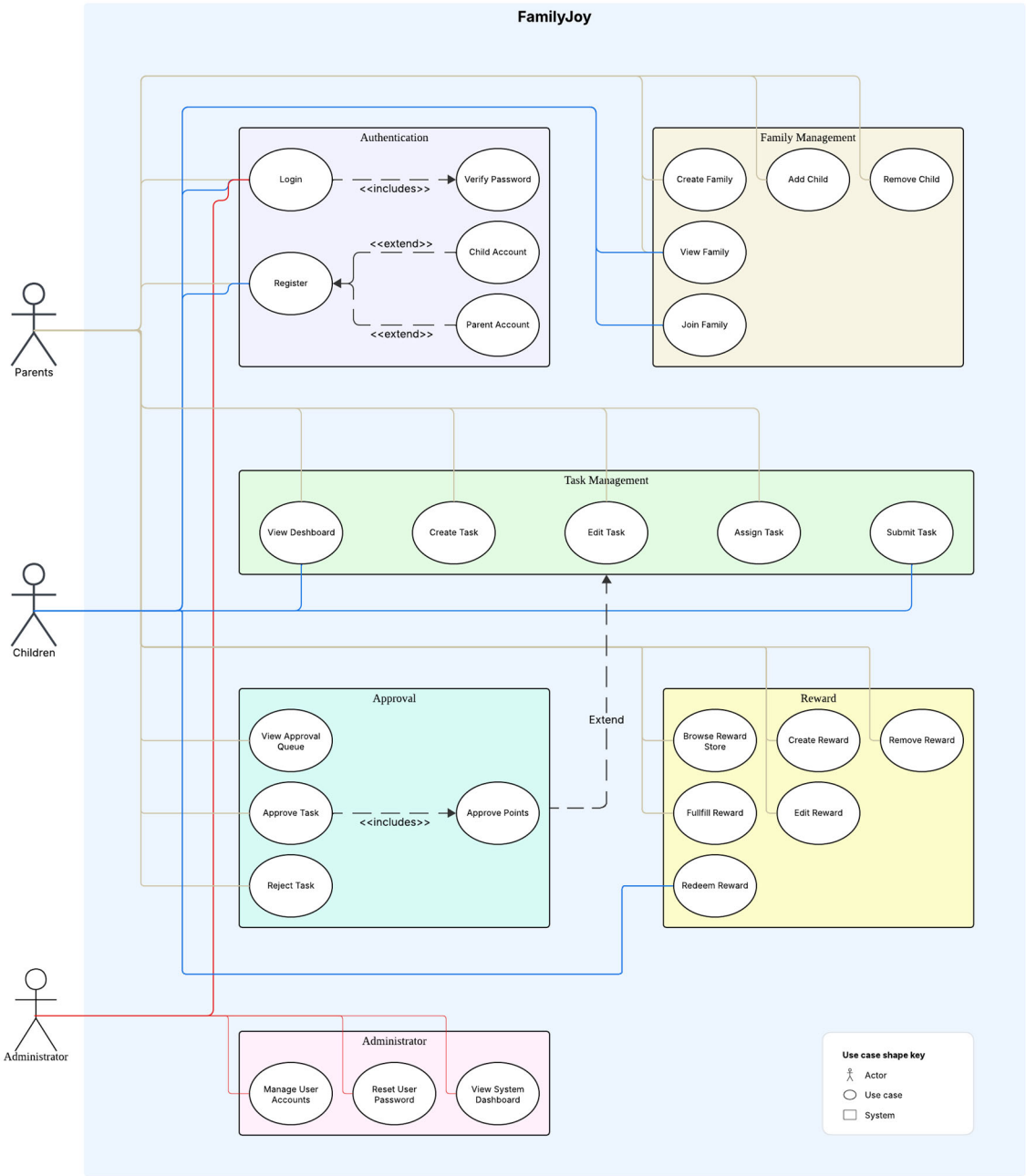
Scrum's time-boxed cycles enable frequent feedback, early risk identification, and continuous value delivery. This validates family needs assumptions and ensures a user-centric product, while demonstrating professional project management.

# 6. PROJECT TIMELINE

| Phase | Main Tasks | Responsible | Start Date | End Date | Status | Output/Value |
|---|---|---|---|---|---|---|
| Sprint 0 | Requirements Analysis & Planing | All Members | 29/09/2025 | 16/11/2025 | Done | Product Backlog, Technical Specification |
| | 0.1 User Roles & Stakeholde Analysis | Project Manager | 29/09/2025 | 4/10/2025 | Done | Requirements Document |
| | 0.2 User Stories & Acceptane Criteria | Product Owner | 6/10/2025 | 13/10/2025 | Done | User Stories Document |
| | 0.3 Project Proposal Submison | All Members | 10/11/2025 | 10/11/2025 | Done | Approved Proposal |
| | 0.4 Development Environmet Setup | Development Team | 11/11/2025 | 16/11/2025 | Planned | Ready Development Environment |
| Sprint 1 | Authentication & Family Foudation | Development Team | 17/11/2025 | 28/11/2025 | Planned | Core System Operational |
| | 1.1 System Architecture Degn | Technical Lead | 17/11/2025 | 19/11/2025 | Planned | System Architecture |
| | 1.2 Database Schema Desig | Backend Developer | 17/11/2025 | 21/11/2025 | Planned | Database Structure |
| | 1.3 Implement Log In API | Backend Developer | 20/11/2025 | 25/11/2025 | Planned | Authentication API |
| | 1.4 Implement Register Paret Account | Backend Developer | 20/11/2025 | 26/11/2025 | Planned | Parent Registration |
| | 1.5 Develop Authentication rontend | Frontend Developer | 24/11/2025 | 28/11/2025 | Planned | Login/Register UI |
| | 1.6 Implement Create Famil Group | Backend Developer | 24/11/2025 | 28/11/2025 | Planned | Family Management API |
| | 1.7 Implement Admin Authetication | Backend Developer | 24/11/2025 | 28/11/2025 | Planned | Admin Login System |
| Sprint 2 | Family Management & Task oundation | Development Team | 1/12/2025 | 12/12/2025 | Planned | Complete Family System |
| | 2.1 Implement Generate Fay Code | Backend Developer | 1/12/2025 | 3/12/2025 | Planned | Code Generation API |
| | 2.2 Implement Register Chil Account | Backend Developer | 1/12/2025 | 5/12/2025 | Planned | Child Registration |
| | 2.3 Implement Join Family | Backend Developer | 4/12/2025 | 6/12/2025 | Planned | Family Join API |
| | 2.4 Develop Family Manageent Frontend | Frontend Developer | 1/12/2025 | 10/12/2025 | Planned | Family UI |
| | 2.5 Implement Create Task PI | Backend Developer | 7/12/2025 | 10/12/2025 | Planned | Task Creation API |
| | 2.6 Develop Basic Task Fron nd | Frontend Developer | 9/12/2025 | 12/12/2025 | Planned | Task Management UI |
| Sprint 3 | Task Management System | Development Team | 15/12/2025 | 26/12/2025 | Planned | Full Task Workflow |
| | 3.1 Implement Assign Task | Backend Developer | 15/12/2025 | 17/12/2025 | Planned | Task Assignment API |
| | 3.2 Implement Edit Task | Backend Developer | 15/12/2025 | 19/12/2025 | Planned | Task Editing API |
| | 3.3 Develop View Task Dashoard | Frontend Developer | 15/12/2025 | 23/12/2025 | Planned | Child Task Dashboard |
| | 3.4 Implement Submit Task r Approval | Backend Developer | 18/12/2025 | 23/12/2025 | Planned | Task Submission API |
| | 3.5 Develop Task Submissio Frontend | Frontend Developer | 22/12/2025 | 26/12/2025 | Planned | Task Completion UI |
| Sprint 4 | Approval & Points System | Development Team | 29/12/2025 | 9/01/2026 | Planned | Complete Approval Workflow |
| | 4.1 Implement Review Apprval Queue | Backend Developer | 29/12/2025 | 2/01/2026 | Planned | Approval Queue API |
| | 4.2 Implement Approve/Reject Task | Backend Developer | 3/01/2026 | 7/01/2026 | Planned | Approval Logic |
| | 4.3 Implement Award Point | Backend Developer | 3/01/2026 | 7/01/2026 | Planned | Points System |
| | 4.4 Develop Approval Queu Frontend | Frontend Developer | 29/12/2025 | 9/01/2026 | Planned | Parent Approval UI |
| | 4.5 Develop Points Display | Frontend Developer | 5/01/2026 | 9/01/2026 | Planned | Points Balance UI |
| Sprint 5 | Rewards System & Admin Cre | Development Team | 12/01/2026 | 23/01/2026 | Planned | Complete Reward Economy + Admin Core |
| | 5.1 Implement Create Rewa | Backend Developer | 12/01/2026 | 16/01/2026 | Planned | Reward Management API |
| | 5.2 Implement Browse Rewd Store | Backend Developer | 12/01/2026 | 16/01/2026 | Planned | Reward Catalog API |
| | 5.3 Implement Redeem Rewrd | Backend Developer | 17/01/2026 | 20/01/2026 | Planned | Redemption System |
| | 5.4 Implement Fulfill Rewar | Backend Developer | 17/01/2026 | 20/01/2026 | Planned | Fulfillment Tracking |
| | 5.5 Develop Reward Managment Frontend | Frontend Developer | 12/01/2026 | 20/01/2026 | Planned | Reward Admin UI |
| | 5.6 Develop Reward Store Fntend | Frontend Developer | 19/01/2026 | 23/01/2026 | Planned | Reward Store UI |
| | 5.7 Implement Manage UseAccounts API | Backend Developer | 12/01/2026 | 16/01/2026 | Planned | User Management API |
| | 5.8 Implement Reset User Pssword | Backend Developer | 17/01/2026 | 19/01/2026 | Planned | Password Reset System |
| | 5.9 Develop Admin User Maagement Frontend | Frontend Developer | 19/01/2026 | 23/01/2026 | Planned | Admin User Interface |
| Sprint 6 | Admin Dashboard & System olish | Development Team | 26/01/2026 | 6/02/2026 | Planned | Production Ready System with Admin |
| | 6.1 End-to-End Integration Tsting | QA Engineer | 26/01/2026 | 30/01/2026 | Planned | Integration Test Report |
| | 6.2 Implement View System Dashboard API | Backend Developer | 26/01/2026 | 30/01/2026 | Planned | System Metrics API |
| | 6.3 Develop Admin Dashboad Frontend | Frontend Developer | 2/02/2026 | 6/02/2026 | Planned | Admin Dashboard UI |
| | 6.4 Performance Optimizatin | Backend Developer | 26/01/2026 | 3/02/2026 | Planned | Optimized System |
| | 6.5 UI/UX Polish | Frontend Developer | 2/02/2026 | 6/02/2026 | Planned | Refined User Interface |
| | 6.6 Security Hardening | Backend Developer | 2/02/2026 | 6/02/2026 | Planned | Secure Deployment |
| Test & Delivery | Final Validation & Deploymet | All Members | 9/02/2026 | 22/02/2026 | Planned | Live Application with Admin |
| | 7.1 User Acceptance Testing | QA Engineer + Team | 9/02/2026 | 18/02/2026 | Planned | UAT Sign-off |
| | 7.2 Production Deployment | Backend Developer | 16/02/2026 | 19/02/2026 | Planned | Live Environment |
| | 7.3 Documentation Finalizabn | All Members | 16/02/2026 | 20/02/2026 | Planned | User & Technical Docs |
| | 7.4 Final Project Delivery | All Members | 22/02/2026 | 22/02/2026 | Milestone | Project Completion |


Gant
0
7
14
21
28
35
42
49
56
63
70
77
84
91
98
105
112
119
126
133
140
147
0.1 User Roles & Stakeholder Analysis
0.2 User Stories & Acceptance Criteria
0.3 Project Proposal Submission
0.4 Development Environment Setup
1.1 System Architecture Design
1.2 Database Schema Design
1.3 Implement Log In API
1.4 Implement Register Parent Account
1.5 Develop Authentication Frontend
1.6 Implement Create Family Group
1.7 Implement Admin Authentication
2.1 Implement Generate Family Code
2.2 Implement Register Child Account
2.3 Implement Join Family
2.4 Develop Family Management Frontend
2.5 Implement Create Task API
2.6 Develop Basic Task Frontend
3.1 Implement Assign Task
3.2 Implement Edit Task
3.3 Develop View Task Dashboard
3.4 Implement Submit Task for Approval
3.5 Develop Task Submission Frontend
4.1 Implement Review Approval Queue
4.2 Implement Approve/Reject Task
4.3 Implement Award Points
4.4 Develop Approval Queue Frontend
4.5 Develop Points Display
5.1 Implement Create Reward
5.2 Implement Browse Reward Store
5.3 Implement Redeem Reward
5.4 Implement Fulfill Reward
5.5 Develop Reward Management Frontend
5.6 Develop Reward Store Frontend
5.7 Implement Manage User Accounts API
5.8 Implement Reset User Password
5.9 Develop Admin User Management Frontend
6.1 End-to-End Integration Testing
6.2 Implement View System Dashboard API
6.3 Develop Admin Dashboard Frontend
6.4 Performance Optimization
6.5 UI/UX Polish
6.6 Security Hardening
7.1 User Acceptance Testing
7.2 Production Deployment
7.3 Documentation Finalization
7.4 Final Project Delivery

# 7. Diagrams


FamilyJoy
Authentication
Login
Verify Password
<<includes>>
<<extend>>
Child Account
Register
<<extend>>
Parent Account
Family Management
Create Family
Add Child
Remove Child
View Family
Join Family
Parents
Task Management
View Deshboard
Create Task
Edit Task
Assign Task
Submit Task
Children
Approval
View Approval Queue
Approve Task
<<includes>>
Approve Points
Reject Task
Extend
Reward
Browse Reward Store
Create Reward
Remove Reward
Fullfill Reward
Edit Reward
Redeem Reward
Administrator
Administrator
Manage User Accounts
Reset User Password
View System Dashboard
Use case shape key
Actor
Use case
System

# 8. References

Figure 1. (2019). Image from How Scrum helped turn around our development process.

Retrieved from https://medium.com/@jw207427/how-scrum-help-turn-around-our-development-process-dac6ff7c700